

ROSGM: A Real-Time GPU Management Framework with Plug-In Policies for ROS 2

Ruoxiang Li¹, Tao Hu², Xu Jiang², Laiwen Li², Wenxuan Xing², Qingxu Deng², Nan Guan¹

¹City University of Hong Kong, Hong Kong SAR

²Northeastern University, China

Abstract—Robot Operating System (ROS) is a prevailing software framework for robotic application development. Graphics Processing Unit (GPU) is widely used in many ROS applications as a first-order computation resource. Unfortunately, ROS does not do any resource management for GPU, and different components in a ROS application directly submit their GPU workload without coordinating with each other, which may cause severe problems in both general performance and real-time capability. This paper presents ROSGM, a real-time ROS 2 GPU Management framework. Instead of providing a fixed GPU management policy for all scenarios, ROSGM allows the addition of any management policy as a plug-in and dynamic switching among different management policies at run-time, which is helpful since GPU management policies are typically device-dependent, and different applications or the same application in different modes may need different GPU management policies. Besides, ROSGM supports dynamic task loading and unloading for integrating additional functionalities when required at run-time. We conduct experiments to evaluate ROSGM. The results show that by properly managing the GPU resource using ROSGM, we can significantly improve the performance of ROS 2 applications. The flexibility of adding management policies as plug-ins, dynamic switching of management policies, and dynamic task loading and unloading helps improve the adaptability of ROSGM.

I. INTRODUCTION

Robot Operating System (ROS) [1] is one of the most popular software frameworks for building robotic applications on various platforms. Its second-generation ROS 2 [2] makes a significant improvement in its performance, reliability, and real-time performance. In ROS 2 applications, multiple *callbacks* perform the execution of different tasks. The *executor* schedules the execution of callback workloads in the threads of the underlying OS [3]–[5], which plays the role of CPU resource manager in ROS 2.

GPU is increasingly used in ROS 2 applications, especially in autonomous machines with computation-demanding functions such as perception, prediction, planning, and intelligent control [6]–[9]. These applications not only require maximizing the resource usage of GPU but also usually need to meet different real-time performance requirements. The management of GPU resources is as important as (and sometimes even more important than) the management of CPU resources for both general and real-time performance of ROS 2 applications.

Unfortunately, the ROS 2 executor does not do any management of GPU. The applications' GPU workloads are directly

sent to GPU for execution, after which their execution depends on the low-level management on the GPU driver level, which appears to be a black box to application developers on the ROS level. Theoretically, one can manage the GPU resource by carefully coordinating the callback scheduling on the ROS level, the thread scheduling on the underlying OS level, and the low-level management on the GPU driver level. However, this contradicts the philosophy of ROS 2, which is to provide a high-level development framework to domain experts who do not have knowledge of the underlying software and hardware. Moreover, after a callback sends the GPU workload to GPU for execution, it will continuously occupy its running thread, waiting for results returned from GPU until the GPU computation completes. This can prolong the execution time of the whole callback chain, especially when using the single-threaded executor, which seriously degrades the application's real-time performance.

To address the above problems, this paper presents ROSGM, a real-time ROS 2 GPU Management framework. We claim it is not wise to use a fixed GPU management policy in ROS 2 to serve all scenarios. From the hardware perspective, different GPU devices have different architectures, and GPU resource management policy is usually hardware-dependent. A management policy relying on specific architecture features [8], [10], [11] may not work on other GPU devices without these features. From the application perspective, different ROS 2 applications may need different GPU management policies. For example, some applications only concern the long-term throughput and thus need a policy to maximize GPU resource utilization. In contrast, some applications may contain tasks with strong real-time requirements and thus need a policy to prioritize the real-time tasks to execute on GPU. Therefore, instead of building a fixed GPU management policy into ROS 2 to serve all applications for all scenarios, ROSGM allows to add any GPU management policy as a plug-in. We implement three GPU management policies as plug-ins for demonstration.

ROSGM allows an application to dynamically switch among different management policies at run-time. This is useful because an application may need different policies when executing in different modes. For example, an autonomous robot application may work in either the normal mode or the urgent mode. In the normal mode, the application requires maximizing the GPU resource utilization, while in the urgent mode, some task handling urgent events needs to gain the

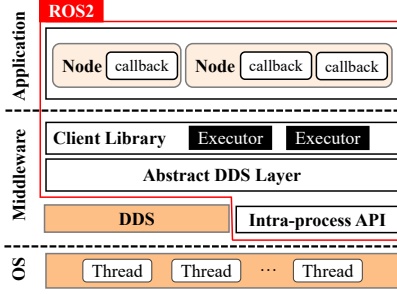


Fig. 1. ROS 2 architecture.

highest priority to access all computation resources and thus can finish as soon as possible. Moreover, ROSGM also allows to dynamically load and unload ROS 2 tasks at run-time under the management of ROSGM. In this way, the user can integrate any additional functionalities when required into an application to satisfy different requirements without stopping the application. Any functionality that may not be needed at some time can also be unloaded dynamically to achieve efficient memory resource utilization, which can be critical, especially for robotic devices with limited memory.

We conduct experiments to evaluate ROSGM, which show that by properly managing the GPU resource using ROSGM, we can significantly improve the performance of ROS 2 applications containing GPU computation tasks. Moreover, the flexibility of adding different management policies as plug-ins can satisfy the different needs of different applications. The dynamic mechanism for switching among different management policies and task loading and unloading can better meet the different requirements of the application when executing under various scenarios.

II. PRELIMINARY

ROS 2 Callback and Executor. ROS 2 (as illustrated in Fig. 1) helps build robotic applications with its software libraries and tools. A ROS 2 application is an executable program for a robotic system. It may contain one or more ROS 2 *nodes*, which are used to perform different tasks. Each node is responsible for a single task/purpose (e.g., object detection, localization, tracking). Different nodes can communicate with each other via topics, services, actions, or parameters. A ROS 2 node (i.e., a class in C++) can contain one or more *callbacks*. Each callback is a function (i.e., a method in C++) containing the codes that realize part of the node’s functionality.

In ROS 2, each callback is bound with a specific type of *message*. Different callbacks are connected to each other by passing different types of messages. An ordered sequence of callbacks forms a *callback chain*. The *executor* is responsible for scheduling callbacks based on their priorities, which depend on the callback type and the registration order during the node initialization phase [3]. The executor keeps querying the incoming messages from the low-level layers of ROS 2. It coordinates the execution of the corresponding callbacks in one or more threads of the underlying operating system. It

mainly consists of a *single-threaded* executor [12] that serially executes callbacks, and a *multi-threaded* executor [13] that allocates the callbacks into multiple threads.

The Default GPU Usage in ROS 2. In this paper, we call a callback using GPU with CUDA programming model (the focus of this paper) a *GPU callback*. In a GPU callback, the newly arrived message is converted into the format that can be used by GPU computation first. Then the standard CUDA program is performed. Memory allocation operation is achieved on the CPU and GPU first. Then, the data transfer operation is issued to copy the data from the host memory to the GPU device memory. And the GPU kernel is launched for computation with multiple threads. After the GPU kernel computation completes, the data transfer operation can copy the results to the host memory. Finally, the GPU callback performs the subsequent processing and/or message publishing (referred to as *post-process*). In a GPU callback, the post-process may need the results returned from GPU. The GPU kernel launch operation is always asynchronous, i.e., once the GPU kernel is launched, control returns to the thread immediately, and the asynchronous data transfer operation can also be used. Despite this, the thread executing the GPU callback still has to be blocked at some point, e.g., using CUDA synchronization functions, like *cudaDeviceSynchronize()* or *cudaStreamSynchronize()*, to wait for the results returned from GPU for post-process. This may lead to the issue described in Section III-B. In this paper, we assume there is only one GPU kernel for each GPU callback, and we refer to the above GPU operations (i.e., memory allocation, data transfer, and GPU kernel launch) as *GPU workloads*.

III. MOTIVATION

This section identifies two major problems of the original GPU workload execution in ROS 2, which motivate our work.

A. Problem 1: Uncontrolled GPU Resource Contention

In ROS 2, the executor schedules the execution of callbacks. However, the GPU callbacks directly submit GPU workloads to GPU, which compete for the GPU resource in an uncontrolled manner. There is no way for ROS 2 to prioritize some GPU workloads over others after being dispatched to GPU for execution. Therefore, a latency-sensitive GPU callback can be blocked by other best-effort ones, which leads to poor real-time performance. The uncontrolled GPU resource contention causes real-time performance issues and may lose general performance. Suppose a single GPU workload does not fully utilize the entire GPU, and we could execute more than one GPU workload simultaneously for better resource utilization. However, the parallel execution of multiple GPU workloads may lead to interference with each other because of contentions on the shared resource. Depending on the degree of interference, the benefit of parallel execution of GPU workloads could be large, small, or even negative. Therefore, without proper control, we cannot get the best performance gain with parallel execution of multiple GPU workloads.

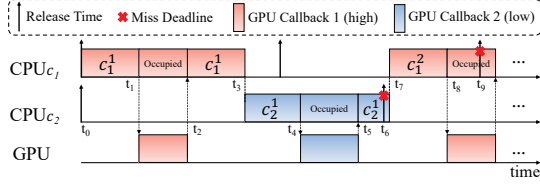


Fig. 2. Execution flow of two GPU callbacks without GPU management.

B. Problem 2: CPU Resource Wasting

As described in Section II, the post-process in a GPU callback may need the result returned from GPU, for which the thread executing the GPU callback has to be blocked at some point to wait for the results returned from GPU for post-process. Therefore, this GPU callback could occupy the thread for a long time and block the execution of other callbacks¹, which may cause (real-time) performance problems for both. This problem is particularly severe when using the single-threaded executor. In the example of Fig. 2, there are two GPU callbacks c_1 (high priority) and c_2 (low priority) scheduled by the single-threaded executor. Assume that the relative deadlines of c_1 and c_2 equal their periods, respectively. At first, c_1^1 and c_2^1 are released simultaneously at time t_0 , and c_1^1 with high priority is scheduled first by the executor. After issuing the GPU operations at time t_1 , the thread will wait for the results returned from GPU for post-process. During the period, c_1^1 occupies the underlying thread without valid execution when its GPU workloads execute on GPU until the result is transferred to the thread at time t_2 , which blocks the execution of c_2^1 with low priority. At time t_3 , c_1^1 completes and c_2^1 is scheduled. Then the same GPU workload execution process is performed for c_2^2 . As a result, both c_2^2 and c_1^2 miss their deadlines at time t_6 and time t_9 , respectively. When using multi-threaded executors, this is also a problem when many callbacks contain GPU workloads (so many threads are blocked simultaneously, and there is no available thread to execute other CPU workloads).

IV. ROSGM

A. Overview

We implement ROSGM and encapsulate it in a ROS 2 node, which provides a series of ROSGM APIs (as listed in Table I) for GPU management in other ROS 2 nodes. It can be instantiated as a node by calling `rosgm_init()` and added to the executor like common nodes. As a ROS 2 node, ROSGM can be seamlessly added into or removed from the ROS 2 workspace as a plug-in, without necessitating modifications to the underlying source code of ROS 2. This is advantageous as it does not compromise the stability of the standard ROS 2 version. By adopting this approach, ROSGM is better equipped to conform to the ROS 2 ecosystem, thereby enabling users to employ it in the same manner as they would with ROS 2.

¹In this case, one GPU callback is blocking the execution of callbacks in other callback chains.

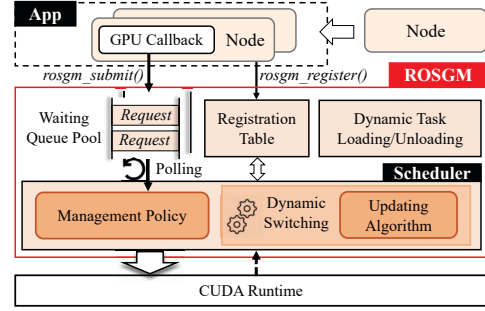


Fig. 3. ROSGM overview.

As illustrated in Fig. 3, ROSGM mainly consists of a registration table, a waiting queue pool, and a scheduler. At the initialization phase, a ROS 2 node containing GPU callbacks registers the GPU workload and the corresponding post-process (namely, a *registration item*) to ROSGM by invoking API `rosgm_register()`. In ROSGM, the registration table (i.e., the *Registration Table* in Fig. 3) maintains all the registration items registered by all nodes containing GPU callbacks in a ROS 2 application. Once the executor schedules a GPU callback, it needs to submit a GPU request (a request for short, i.e., a *Request* in Fig. 3), which contains the newly arrived data². All the submitted requests wait to be scheduled in the waiting queue pool $\Phi = \{Q_{wq} | wq \in [0, M - 1]\}$ with M waiting queues. The waiting queue pool is configurable for users in terms of the number of waiting queues based on the requirement of the management policy. The scheduler executing in a *manager thread* keeps querying the submitted requests from the waiting queue pool. The manager thread is a specialized CPU thread with the highest static priority in the underlying operating system. Its priority is set during the initialization phase of the ROSGM node. This CPU thread operates independently of the ROS 2 executor and is intended to prevent interference from the executor's scheduling strategy. The scheduler is equipped with the management policy to conduct GPU management for ROS 2 application. When there is no request to be scheduled in the waiting queue pool, the manager thread will self-suspend to save CPU cycles. And it will be activated once a request is submitted into the waiting queue pool. For all requests, the scheduler can query the corresponding registration items in the registration table and apply the specific management policy to schedule them.

ROSGM allows adding different GPU management policies as plug-ins into the scheduler and allows an application to dynamically switch among different policies at run-time (Section IV-C). We implement three GPU management policies as demos, including *Exclusive Policy*, *Sharing Policy*, and *Preemption Policy* (Section V). We enable ROSGM to support dynamic task loading and unloading (Section IV-D). When required, this mechanism allows the user to dynamically inte-

²From this perspective, we design ROSGM as a data-driven framework, which is explained in the following subsection IV-B.

TABLE I
ROSGM APIs

<i>rosgm_init()</i>	initializes ROSGM to configure the management policy and the GPU request submission method (i.e., synchronous or asynchronous).
<i>rosgm_register()</i>	registers the GPU workload and the corresponding post-process (post-process can be empty for synchronous GPU request submission method) of a GPU callback to the registration table in ROSGM.
<i>rosgm_deregister()</i>	removes a previously registered GPU workload and the corresponding post-process (if any) from the registration table in ROSGM.
<i>rosgm_submit()</i>	submits a GPU request to ROSGM (depending on whether the synchronous or asynchronous GPU request submission method is configured, this function will block the thread calling it or not).

grate any additional functionalities into an application without stopping to satisfy different requirements. The dynamically loaded tasks will be scheduled by the same executor that schedules the executing tasks of the running application, and ROSGM will manage all the newly loaded GPU workloads. Furthermore, ROSGM provides two GPU request submission methods, i.e., asynchronous and synchronous methods (Section IV-E). The asynchronous method can solve the problem of CPU occupation during GPU workload execution, i.e., Problem 2 in Section III-B. Meanwhile, the synchronous method can achieve a similar or even the same performance as the asynchronous method with the proper setting for the executor. We provide the evaluation analysis comparing the asynchronous and synchronous methods in section VI-C.

To use ROSGM, the user can select the management policy and the GPU request submission method (i.e., asynchronous method or synchronous method) by calling *rosgm_init()*.

B. Data-driven Framework

In ROS 2 applications, callbacks usually are triggered periodically. To reduce the overhead for message transfer between the GPU callbacks and ROSGM, we only register the information once that remains constant for GPU workload execution. And the periodically updated data in the GPU callback for GPU kernel computation can be submitted to ROSGM every time the GPU callback is invoked with the updated data. As illustrated in Fig. 3, in a ROS 2 node, if there is a GPU callback, the node registers the registration item to the registration table at the initialization phase. This API also passes a unique *id* for each GPU callback identifying the registration item in the registration table. Each row in the registration table is a tuple in the form of $(id, workload, postprocess, state)$, where the corresponding *id* is the unique identifier for a GPU callback (and the corresponding registration). For each registration item, the registration table also holds its *state* (*active* or *inactive*). While *active* indicates that the scheduler has scheduled the corresponding request and the corresponding GPU workload is executing currently, *inactive* state represents that no corresponding request has been scheduled for GPU workload execution. Every time a GPU callback is scheduled by the executor, it only needs to submit a *request* to ROSGM rather than the whole registration item. A request is a tuple $(id, t, data)$, including the *id* of the GPU callback indicating the corresponding registered registration item, the timestamp indicating its submission time, and the newly updated data for the GPU kernel computation, respectively. The user can

deregister a registration item from the registration table in ROSGM by calling API *rosgm_deregister()*.

C. Management Policy as Plug-In

1) *Addition of Management Policy*: As stated previously, different ROS 2 applications may execute on different GPU devices with various architectural features. Even the same application may need different GPU management policies in different execution modes when facing different scenarios. Hence, the management framework requires various GPU management policies to meet different requirements and goals. To improve the adaptability and flexibility of ROSGM, we equip ROSGM with the ability to integrate any management policy as a plug-in. To achieve this, we implement the management policy base class (i.e., an abstract class in C++) into a ROS 2 package as the plug-in *provider*. The base class defines a generic policy class, and a plug-in class is a derived class inheriting the base class, which defines a specific management policy with the user's implementation. When implementing a plug-in class, the user should invoke the macro (i.e., *PLUGINLIB_EXPORT_CLASS*) to register the plug-in class to be discovered. The arguments of the macro identify the type of the plug-in class (e.g., *policy_plugins::UserPolicy*) and the type of the base class (i.e., *policy_base::PolicyBase*). The plug-in class that implements a specific management policy holds the required number of the waiting queue, i.e., M . It also contains a *mapping table* $\Theta = \{(id, wq) | id \in [0, N-1], wq \in [0, M-1]\}$, where N is the number of the GPU callbacks. The mapping table describes the mapping relation (id, wq) between the GPU callback c_{id} and the corresponding waiting queue $Q_{wq} \in \Theta$. The mapping table is configured at the initialization phase of the plug-in class based on its configuration file. It is also significant during the dynamic switching of management policy, which is used in the waiting queue pool updating algorithm, i.e., Algorithm 1.

By inheriting the base class in the plug-in provider, users can easily implement their management policy plug-ins and then apply them in ROSGM to meet different needs. One can configure any implemented management policy to ROSGM at the initialization phase when calling *rosgm_init()*. To demonstrate this plug-in addition mechanism, we implement three demo management policies described in Section V.

2) *Dynamic Switching of Management Policy*: Moreover, we enable ROSGM with dynamic switching of management policies at run-time. The dynamic switching mechanism is implemented using the ROS 2 pub-sub communication method. A subscription callback in ROSGM subscribes to the topic

Algorithm 1: Waiting queue pool updating algorithm.

```

1 if  $\overline{M} > M$  then
2    $\text{add } Q_i \text{ to } \Phi, i \in [M, \overline{M} - 1]$ ;
3 for each mapping  $(id, wq)$  in  $\Theta$  do
4   find  $\overline{wq}$  satisfying  $(id, \overline{wq}) \in \overline{\Theta}$  with the same  $id$ ;
5   if  $\overline{wq} \neq wq$  then
6     for each request in  $Q_{wq}$  do
7       if request. $id = id$  then
8         delete the request from  $Q_{wq}$  and insert
          it into  $Q_{\overline{wq}}$  according to request. $t$ ;
9 if  $\overline{M} < M$  then
10   $\text{delete } Q_j \text{ from } \Phi, j \in [\overline{M}, M - 1]$ ;

```

rosgm_plugin. The user can use the ROS 2 command line *ros2 topic pub* to publish a *string* type message indicating the selected policy type (e.g., *policy_plugins::UserPolicy*) to the topic *rosgm_plugin*. When the subscription callback is triggered, it notifies the manager thread to wait and switches the management policy currently used in ROSGM.

Furthermore, different management policies may have different requirements for the waiting queue pool regarding the number of waiting queues. For example, as introduced in Section V, *Exclusive Policy* needs two waiting queues, one for the request with the privilege and one for the others, and *Sharing Policy* needs only one waiting queue for all submitted requests. Hence, ROSGM has to support updating the waiting queue pool when the dynamic switching of management policy occurs. On the one hand, the waiting queue pool should be updated into a new pattern satisfying the newly selected management policy requirement. And the remaining requests in the waiting queue pool should be re-arranged to the updated waiting queue pool based on the need of the newly selected management policy. Then the remaining requests can be scheduled under the newly selected management policy. The requests that have been launched to GPU will keep the previously selected management policy.

We propose the waiting queue pool updating algorithm (Algorithm 1) for the dynamic switching of management policy. Assume that before switching, the mapping table of the management policy is Θ , and the number of the required waiting queues is M . After switching, the mapping table of the new management policy is $\overline{\Theta}$ and the number of the required waiting queues is $\overline{M}$. Algorithm 1 first adds $\overline{M} - M$ waiting queues to Φ if $\overline{M} > M$. Then for each mapping $(id, wq) \in \Theta$, it gets $(id, \overline{wq}) \in \overline{\Theta}$ with the same id . If they are the same, i.e., $\overline{wq} = wq$, the corresponding requests from c_{id} will remain in Q_{wq} . Otherwise, all requests from c_{id} will be removed from Q_{wq} . And they will be inserted into $Q_{\overline{wq}}$ according to their timestamps based on the binary search algorithm. Finally, the algorithm deletes $M - \overline{M}$ waiting queues from Φ if $\overline{M} < M$.

Example. Assume that an application has four GPU callbacks c_0, c_1, c_2 , and c_3 . Before switching, the management

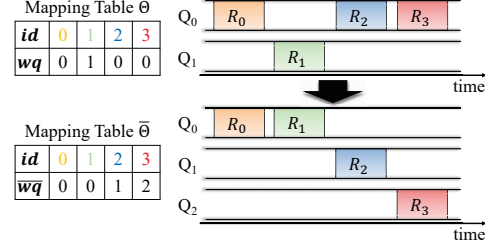


Fig. 4. An illustration example for the waiting queue pool updating algorithm.

policy holds the mapping table Θ and configures two waiting queues (i.e., Q_0 and Q_1) in the waiting queue pool. After switching, the newly selected management policy holds the mapping table $\overline{\Theta}$ and configures three waiting queues (i.e., Q_0, Q_1 , and Q_2) in the waiting queue pool. Assume that the waiting queue status is shown in the top right of Fig. 4 before switching the management policy. There are three requests, i.e., R_0, R_2 , and R_3 , in Q_0 , which are from the GPU callbacks c_0, c_2 and c_3 , respectively. One request R_1 from c_1 is in Q_1 . When switching the management policy, Algorithm 1 adds a new waiting queue Q_2 to the waiting queue pool. Then it checks each mapping $(id, wq) \in \Theta$ and the corresponding mapping $(id, \overline{wq}) \in \overline{\Theta}$. When $id = 0$, $wq = \overline{wq}$. So R_0 remains in Q_0 . When $id = 1$, $wq \neq \overline{wq}$ and R_1 is removed from Q_1 and inserted into Q_0 after R_0 according to their timestamps. When $id = 2$, $wq \neq \overline{wq}$ and R_2 is removed from Q_0 and inserted into Q_1 . When $id = 3$, $wq \neq \overline{wq}$ and R_3 is removed from Q_0 and inserted into Q_2 . After switching, the waiting queue status is illustrated in the bottom right of Fig. 4.

D. Dynamic Task Loading and Unloading

Considering that some tasks are not needed at a certain execution phase of an application, these tasks may not be linked at compile time to save memory resources. One can dynamically load them when it is necessary to execute these tasks and unload them when they are not required. We implement ROSGM to support dynamic task loading and unloading tasks at run-time. In ROS 2, we can implement a node as a *component* that is built as a shared library. These components can be registered in the ROS 2 workspace, allowing them to be discovered by other packages. Additionally, components can be dynamically loaded or unloaded at runtime, enabling the execution of a ROS 2 application with dynamic loading and unloading capabilities.

The dynamic task loading and unloading mechanism in ROSGM works based on the ROS 2 service-client communication method. To use ROSGM, one can implement a task as a common node, which can only be loaded into ROSGM at compile time. For the tasks implemented as components, ROSGM can also load and unload them at run-time, which will be scheduled by the same executor that schedules the running nodes/components. When unloading a component, the corresponding registration items registered by it will be

unregistered from the registration table. It means that the requests from its GPU callbacks will become invalid and will also be removed from the waiting queue pool. If a task is unloaded while its GPU request is executing on the GPU, the corresponding request can continue execution without being affected by the task unloading until the GPU workload is completed. Nevertheless, the post-process associated with the request will not be initiated, resulting in the absence of any message publication based on the GPU execution result to the subsequent topic.

E. GPU Request Submission

To deal with the problem of CPU occupation during GPU workload execution, we use an asynchronous GPU request submission method (referred to as *asynchronous method* for simplicity) in ROSGM. With the asynchronous method, the thread executing a GPU callback submits requests to ROSGM by calling `rosgm_submit()`. Once the command completes, the control returns to the thread immediately. If there is no other command, the GPU callback will end its execution, and the thread will be released. When a request is selected by the ROSGM scheduler to execute on the GPU, the corresponding *post-process*, which has been previously registered in the registration table, can be accessed with the same *id* as the request. And the *post-process* will be registered based on the CUDA runtime API `cudaLaunchHostFunc()`, which allows the post-process to be performed after the completion of the preceding GPU workloads in the same stream without requiring synchronization with the CPU. When the GPU kernel execution completes, CUDA run-time will automatically create a new CPU thread for executing the corresponding post-process, which further processes the GPU kernel execution results and publishes the processed message to the specific topic. The asynchronous method can solve the problem of CPU occupation during the GPU workload execution. This is because a thread executing the GPU callback will be released immediately once the request has been submitted to ROSGM. So the thread does not need to wait for the GPU workload to complete, and it can be utilized for executing the other callbacks. Based on the asynchronous method, ROSGM can achieve parallel execution of CPU and GPU workloads, which improves the real-time performance of ROS 2 applications containing GPU callbacks.

An illustration example. As shown in Fig. 5, the same ROS 2 application as in Fig. 2 executes with the GPU callbacks c_1 and c_2 under the management of ROSGM. The scheduler runs in a CPU thread (indicated as CPU_{ROSGM} in Fig. 5). Under the asynchronous method, c_1^1 submits a request to ROSGM at time t_1 . After that, the thread executing c_1^1 is released, and c_2^1 is scheduled for execution by the single-threaded executor. Meanwhile, since the request is submitted to the waiting queue, the scheduler in ROSGM schedules the corresponding GPU workload to execute on the GPU device with the data in the request at time t_2 . When the workload execution completes, the post-process will be invoked at time t_3 by CUDA run-time to execute on a CPU thread, which

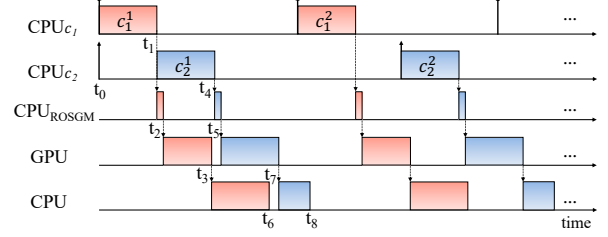


Fig. 5. Execution flow of two GPU callbacks managed by ROSGM.

publishes the message to a specific topic and completes at time t_6 . Besides, at time t_4 , c_2^1 submits a request to ROSGM, which is scheduled for GPU workload execution at time t_5 . The corresponding GPU workload and post-process are complete at time t_7 and t_8 , respectively. Compared with the execution flow without the management of ROSGM in Fig. 2, both callbacks can meet their deadlines and achieve shorter response time, i.e., better real-time performance.

In addition, ROSGM also provides a synchronous GPU request submission method (referred to as *synchronous method* for simplicity). For the synchronous method, the thread executing the GPU callback submits requests to ROSGM using `rosgm_submit()`. Then, it will be blocked until the GPU workload execution completes on the GPU device. The thread will be awakened by CUDA run-time. After that, it processes the GPU execution result and publishes the message to a specific topic. If a request is preceded by multiple other requests in the waiting queue, the request will wait for scheduling until all the preceding requests have been scheduled.

Considering the execution process outlined above, executing GPU callbacks under the synchronous method closely resembles the default GPU callback execution in ROS 2. However, the synchronous method enables ROSGM to manage GPU workloads using specific GPU management policies. When using the single-threaded executor, the synchronous method can still lead to Problem 2, where the CPU is occupied during GPU workload execution, as described in Section III-B. This problem persists even when using the multi-threaded executor and impacts the real-time performance of the application. Nevertheless, the synchronous method can achieve comparable or identical performance to the asynchronous method when the number of threads used by the multi-threaded executor is greater than or equal to the number of callback chains in the application. Additionally, the synchronous method is more straightforward to use since it can execute the post-process in the thread executing the GPU callback, thereby eliminating the need for users to register the post-process to ROSGM while calling `rosgm_register()`. For these reasons, we provide both asynchronous and synchronous GPU submission methods in ROSGM, allowing users to choose the method that best suits their needs. In Section VI-C, we provide an evaluation analysis comparing the asynchronous and synchronous methods for reference.

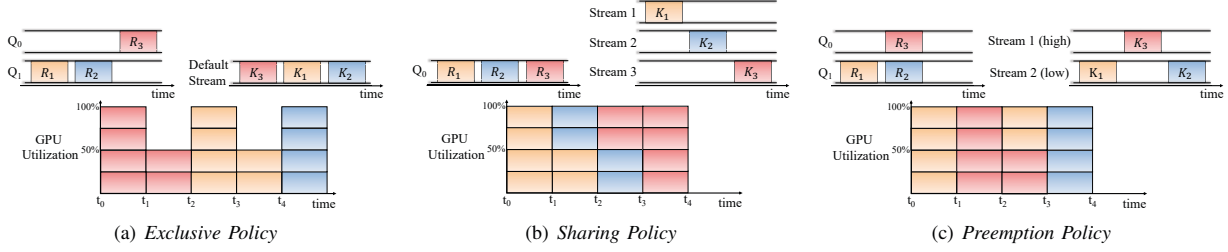


Fig. 6. Three demo management policies in ROSGM. For simplicity, we omit the data transfer operations between CPU and GPU. The length of each interval in (a), (b), and (c), such as $[t_0, t_1]$, depends on the execution time of the corresponding kernel.

V. THREE DEMO PLUG-INS IN ROSGM

ROSGM allows the addition of any GPU management policy as a plug-in. In this work, we implement three GPU management policies as shown in Fig. 6 for demonstration: *Exclusive Policy*, *Sharing Policy*, and *Preemption Policy*.

A. Exclusive Policy

In the CUDA programming model, CUDA stream [14] contains a sequence of issued commands that execute in order. It can be classified into default stream (i.e., NULL stream) and non-default stream (i.e., user-defined stream). GPU kernels in the default stream are dispatched to the GPU device in a FIFO manner. Only after the previous kernel completes, the later kernel can be dispatched. ROSGM provides the *Exclusive Policy*, implemented with two waiting queues (i.e., Q_0 and Q_1) based on the default stream. When using this policy, the user can assign the *privilege* (i.e., the highest priority) to a specific GPU callback, which can be latency-sensitive. And the privilege will also be assigned to the corresponding requests from this GPU callback. The requests from the GPU callback with the privilege are put into Q_0 , while others are put into Q_1 . If a request exists in Q_0 , the scheduler takes it first whether or not the request was submitted later than the other requests in Q_1 . In this way, any other requests in Q_1 will wait for the request with the privilege to complete. Otherwise, if there is no submitted request with privilege in Q_0 or the privilege is not assigned to any GPU callback, the scheduler takes the requests from Q_1 in a FIFO manner. And the scheduler dispatches the corresponding GPU operations to the default stream for execution for each request.

Example 1. Assume that a ROS 2 application has three GPU callbacks that submit requests R_1 , R_2 , and R_3 to ROSGM, respectively. The corresponding GPU kernels are K_1 , K_2 , and K_3 . As shown in the top left of Fig. 6-(a), at some point, the waiting queue status is that two requests R_1 and R_2 are in Q_1 and a request R_3 is in Q_0 , which is submitted later than R_1 and R_2 . The scheduler configured with *Exclusive Policy* takes R_3 first from Q_0 , then R_1 and R_2 from Q_1 . The GPU operations (for simplicity, only kernels K_3 , K_1 , and K_2 are illustrated in the top right of Fig. 6-(a)) corresponding to R_3 , R_1 , and R_2 will be put into the default stream sequentially. And the kernels execute as shown in the bottom of Fig. 6-(a). The kernel K_3 executes first and monopolizes the GPU from

time t_0 to t_2 . The kernel K_1 executes from time t_2 to t_4 , after which the kernel K_2 executes.

B. Sharing Policy

When using the CUDA programming model, we can achieve concurrent GPU operation execution by using multiple non-default streams. For example, two kernels from two different streams can run in parallel if the GPU resources (e.g., thread, registers) are satisfied. ROSGM provides a *Sharing Policy* with one waiting queue Q_0 used. It contains multiple non-default streams with the same priority, for which the number of streams is configurable for users. The scheduler maps every request in Q_0 to a non-default stream and dispatches the corresponding GPU operations into this stream.

Example 2. Assume that the same application is used as in Example 1. As shown in the top left of Fig. 6-(b), at some point, the waiting queue status is three requests R_1 , R_2 , and R_3 are in Q_0 . The scheduler takes R_1 , R_2 , and R_3 from Q_0 successively. The kernels K_1 , K_2 , and K_3 (as illustrated in the top right of Fig. 6-(b)) are put into three different streams. So they can execute concurrently as shown in the bottom of Fig. 6-(b). K_1 executes first, starting at time t_0 . From time t_0 to t_1 , K_1 monopolizes the GPU, and then from time t_1 to t_2 , it shares GPU resources with K_2 and executes in parallel with K_2 . From time t_2 to t_3 , K_2 executes with K_3 in parallel, after which K_3 monopolizes the GPU from time t_3 to t_4 . Furthermore, the parallel execution of two kernels from time t_1 to t_3 may interfere with each other because of contentions on the shared GPU resources (an evaluation example is given in Section VI-B). Hence, *Sharing Policy* should be selected with proper control to avoid unnecessary performance penalties.

C. Preemption Policy

When using the CUDA stream, we can allocate a priority to a non-default stream. Kernels in the high-priority stream will be executed prior to the kernels in the low-priority stream. And they can preempt a running kernel from a low-priority stream at the block-level granularity (i.e., currently running blocks will complete, and new blocks will only be scheduled after the kernels from the high-priority stream are complete) [15]. With the priority strategy, a management policy can provide the preemption ability for ROSGM. In ROSGM, we implement the *Preemption Policy* using two streams with priorities. The

policy maintains two waiting queues, one high-priority waiting queue Q_0 for the latency-sensitive requests and one low-priority waiting queue Q_1 for the best-effort requests. The requests in Q_0 will be scheduled prior to those in Q_1 . The GPU operations corresponding to the requests in Q_0 will be dispatched into the high-priority stream. In contrast, the GPU operations corresponding to the requests in Q_1 will be dispatched into the low-priority stream. Kernels in the high-priority stream can preempt those in the low-priority stream at the block-level granularity. In this way, ROSGM can achieve priority allocation to requests from different GPU callbacks with different real-time requirements by using the *Preemption Policy*.

Example 3. The same application is used as in Example 1. As shown in the top left of Fig. 6-(c), at some point, the scheduler takes a request R_1 from Q_1 , and the corresponding kernel K_1 is put into the stream 2 with low priority. At some point between t_0 and t_1 , K_1 is being executed, and the requests R_2 and R_3 are submitted to ROSGM simultaneously. They are put into Q_1 and Q_0 , respectively. The scheduler takes R_3 from Q_0 first and dispatches K_3 into the high-priority stream 1. After that, it dispatches K_2 into the low-priority stream 2. From time t_0 to t_1 , the blocks of K_1 monopolize the GPU. Until time t_1 , these blocks of K_1 complete, and K_3 preempts the remaining blocks' execution in K_1 . From time t_2 to t_3 , K_3 and K_1 can execute in parallel because K_3 does not occupy all the GPU resources, and the remained resources can be utilized by K_1 .

VI. EXPERIMENTS

In the following, we evaluate the performance of our proposed ROSGM, including the evaluation of GPU resource contention, the evaluation of CPU occupation during GPU workload execution, the performance under the multi-threaded executor, and the run-time overhead of ROSGM.

A. Experiment Setup

We have implemented ROSGM in the Foxy Fitzroy version of ROS 2, running on top of Ubuntu 20.04.5 with kernel 5.13.0-44-generic. The experiment platform is a PC equipped with an NVIDIA Quadro T1000 Mobile GPU (with 14 SMs, each SM executes up to 16 blocks at the same time), an Intel(R) Core(TM) i5-9400H CPU processor (4 cores), and a 24 GB main memory. All the GPU tasks are written in CUDA 11.7 and compiled using NVCC V11.7.64 on top of the NVIDIA 515.48.07 driver.

Our experiments require a suitable benchmark set for ROS 2, but we have not identified one that meets our needs. So, we design a benchmark containing different artificial *micro-kernels* for evaluation. These micro-kernels are configurable in terms of the number of blocks, the number of threads, and the kernel computation time. Table II and III describe the details of two case studies. For simplicity, only timer callbacks and subscription callbacks are used in our experiments. Each callback is characterized by the priority, the type (CPU callback or GPU callback), and the Worst-Case Execution Time (i.e.,

TABLE II
DESCRIPTION OF CALLBACK CHAINS FOR CASE STUDY 1

Chain	Period (ms)	Callback	Priority	Callback Type	WCET (ms)
C_1	100	$c_{1,1}$	1	CPU	5
		$c_{1,2}$	3	GPU	30
		$c_{1,3}$	5	CPU	5
C_2	100	$c_{2,1}$	2	CPU	10
		$c_{2,2}$	4	GPU	30
		$c_{2,3}$	6	CPU	5

TABLE III
DESCRIPTION OF CALLBACK CHAINS FOR CASE STUDY 2

Chain	Period (ms)	Callback	Priority	Callback Type	WCET (ms)
C_1	100	$c_{1,1}$	1	CPU	3
		$c_{1,2}$	5	GPU	8
		$c_{1,3}$	9	CPU	10
C_2	100	$c_{2,1}$	2	CPU	3
		$c_{2,2}$	6	GPU	8
		$c_{2,3}$	10	CPU	10
C_3	100	$c_{3,1}$	3	CPU	3
		$c_{3,2}$	7	GPU	8
		$c_{3,3}$	11	CPU	10
C_4	100	$c_{4,1}$	4	CPU	3
		$c_{4,2}$	8	GPU	8
		$c_{4,3}$	12	CPU	10

WCET). The timer callback, i.e., the *first* callback in a callback chain, has the highest priority. The priorities of other callbacks are set as shown in Table II and III. A callback unrelated to GPU workload execution is classified as a CPU callback. In case studies 1 and 2, we assume that the relative deadline of each callback chain equals its period. With our settings about these callback chains, all of them are schedulable, i.e., the completion time of the last callback execution is always earlier than the deadline. We conduct the experiments and trace the event behavior (e.g., timer callback and subscription callback) with the tracing tool, *ros2_tracing* [16], for further analysis. The observed response time in our experiments was obtained by recording the release time of timer callback execution and the completion time of the last callback execution within the same chain.

B. Evaluation of GPU Resource Contention

In this evaluation, we use the ROS 2 application as shown in Table II. This application has two callback chains C_1 and C_2 . Both C_1 and C_2 contain three callbacks, and $c_{1,2}$ and $c_{2,2}$ are GPU callbacks. The GPU kernels used in $c_{1,2}$ and $c_{2,2}$ are the same, and both have 112 blocks with 64 threads in each block³. Furthermore, these two kernels will compete with the GPU memory resources if they execute on the GPU simultaneously. We evaluate this application in terms of the response time of C_1 and C_2 under two different management policies, i.e., *Exclusive Policy* and *Sharing Policy*. When using

³Up to 224 blocks can simultaneously reside on the GPU we use when other resources are not constrained. So these two kernels can execute concurrently on the GPU.

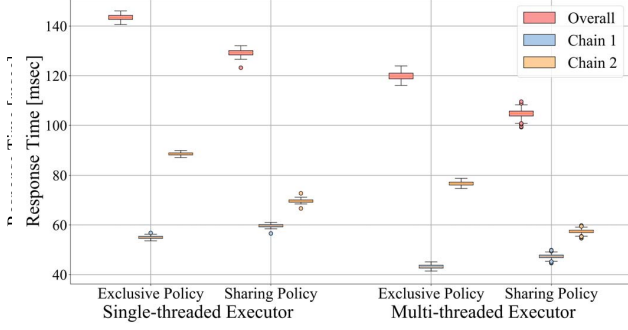


Fig. 7. Observed response time of chains under the management of ROSGM.

Exclusive Policy, $c_{1,2}$ is assigned with the privilege. When using *Sharing Policy*, both the kernels used by $c_{1,2}$ and $c_{2,2}$ (i.e., K_1 and K_2 , respectively) can execute concurrently on the GPU. We conduct 1000 experiments under each management policy, i.e., the timer releases 1000 times, and 1000 response time values are recorded.

Fig. 7 illustrates the observed response time of C_1 and C_2 under the single-threaded and multi-threaded executors. The overall response time is also provided, which is the sum of the response times of C_1 and C_2 . Based on the results in Fig. 7, we know that C_1 achieves a shorter response time under *Exclusive Policy*. The reason is that K_1 can monopolize GPU resources without being subject to resource contention from K_2 . In contrast, the response time of C_1 is longer under *Sharing Policy*. Because the contention of the GPU memory resources between K_1 and K_2 prolongs the execution time of K_1 . However, as shown in Fig. 7, ROSGM using *Sharing Policy* achieves a shorter overall response time. It means that the GPU device achieves higher throughput because K_2 executes in parallel with K_1 and achieves a shorter response time. The above experiment result demonstrates our first motivation (Problem 1), i.e., GPU workloads of different GPU callbacks in ROS 2 application can cause uncontrolled GPU resource contention in an uncontrolled manner. Moreover, ROSGM can provide the proper GPU management by configuring different GPU management policies as required under different requirements. For instance, *Exclusive Policy* should be configured if the application needs a stronger real-time requirement for C_1 , and *Sharing Policy* can be configured if the application requires a higher throughput.

C. Evaluation of CPU Occupation during GPU Workload Execution

We compare the effect of the asynchronous and synchronous methods on ROS 2 applications that contain GPU callbacks, i.e., the CPU occupation during GPU workload execution (Problem 2). In this evaluation, we use the same ROS 2 application (Table II) as in Section VI-B, except we replace the GPU kernels used by $c_{1,2}$ and $c_{2,2}$. Both the GPU kernels have 560 blocks with two threads in each block. The observed response time of C_1 and C_2 are evaluated under the three GPU

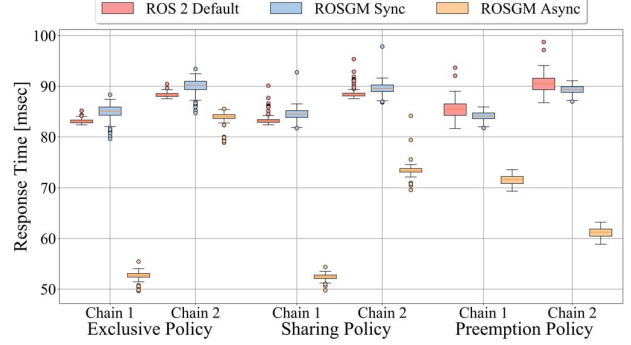


Fig. 8. Observed response time of chains under the single-threaded executor.

request submission methods, i.e., the ROS 2 default method without the management of ROSGM (abbreviated as **ROS 2 Default**), the synchronous method (abbreviated as **ROSGM Sync**), and the asynchronous method (abbreviated as **ROSGM Async**). We conduct 1000 experiments (i.e., 1000 response time values are recorded) for each management and each submission method.

Fig. 8 illustrates the observed response time of C_1 and C_2 under the three submission methods and the single-threaded executor. Based on the experiment results in Fig. 8, we know that the asynchronous method outperforms the other two methods under all management policies. The reason is that the asynchronous method allows the parallel execution of CPU and GPU workloads, which results in higher resource utilization for both CPU and GPU. This yields the reduced response time of C_1 and C_2 .

However, when using the multi-threaded executor (configured with two threads), all three submission methods achieve almost the same results under the three management policies as shown in Fig. 9. It is because the application has two callback chains, and the multi-threaded executor is configured with two threads, so when a thread is occupied by a GPU callback in a callback chain, the second thread can execute other callbacks in another callback chain. For example, $c_{1,2}$ of C_1 will not block the execution of $c_{2,2}$ of C_2 even when the synchronous method is used. The above result suggests that the number of threads should be carefully configured when using the synchronous method. We provide further analysis of the above suggestion in Section VI-D.

Considering the experiments under the asynchronous method in Fig. 8, it also indicates that configuring different management policies in ROSGM can achieve different performance effects, which helps to meet the different requirements of the ROS 2 application. For example, we can select the *Exclusive Policy* and *Sharing Policy* if the real-time requirement of c_1 is the priority. When the application has a real-time requirement for c_1 and needs to get higher throughput, the *Sharing Policy* will be the better choice. If the real-time requirement of c_2 is the priority, the *Preemption Policy* should be selected. The experiment results under the multi-threaded executor in Fig. 9 also give the same conclusion.

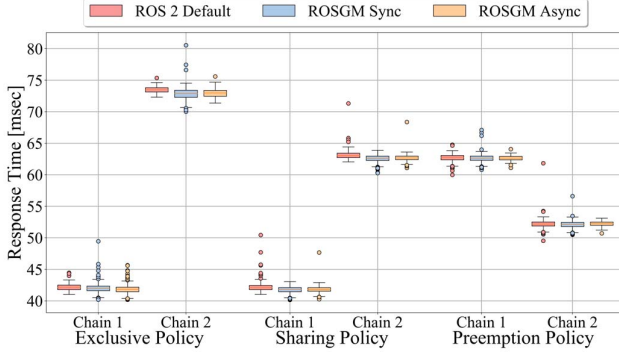


Fig. 9. Observed response time of chains under the multi-threaded executor.

An interesting observation is that not only the scheduling strategy used by the management policy in ROSGM but also the priorities of GPU callbacks can affect the execution of GPU workloads. A GPU callback with higher priority can submit requests to ROSGM earlier than a GPU callback with lower priority, leading to the requests submitted by the GPU callback with higher priority being scheduled earlier by the scheduler in ROSGM. We conduct experiments to demonstrate this observation by using the same setting as in Table II, except that we exchange the priorities of $c_{1,2}$ and $c_{2,2}$, i.e., the priority of $c_{1,2}$ is set as 4 and the priority of $c_{2,2}$ is set as 3. Fig. 10 shows the response times of C_1 and C_2 before and after exchanging the priorities of $c_{1,2}$ and $c_{2,2}$ (i.e., **Case 1** and **Case 2**, respectively) under the single-threaded and multi-threaded executors. In these experiments, ROSGM is configured with the asynchronous method and the *Preemption Policy*. Consider the results under the single-threaded executor in Fig. 10, C_2 achieves shorter response time after exchanging the priorities of $c_{1,2}$ and $c_{2,2}$, i.e., when $c_{2,2}$ has a higher priority than $c_{1,2}$. The reason is that when both $c_{1,2}$ and $c_{2,2}$ are ready to be scheduled by the executor, $c_{2,2}$ with higher priority will be scheduled first. The request from $c_{2,2}$ will be submitted to ROSGM earlier, and the corresponding GPU kernel (i.e., K_2) will be launched earlier to GPU for execution by the scheduler. So K_2 does not need to wait for the running blocks of K_1 to complete. Fig. 10 also shows the experiment result under the multi-threaded executor with two threads. Since there are two threads configured in the executor and $c_{2,1}$ has a shorter WCET than $c_{2,2}$, $c_{2,2}$ will always be scheduled later than $c_{1,2}$, which results in the same performance for both cases. However, the priorities of $c_{1,2}$ and $c_{2,2}$ can still affect the response time of the callback chain if $c_{1,2}$ and $c_{2,2}$ can be ready to be scheduled at the same time in some settings, e.g., the WCET of $c_{2,2}$ equals 5 ms. This observation suggests that we must carefully consider the assignments of priorities for GPU callbacks in ROS 2 applications.

D. Evaluation under Multi-threaded Executor

To better understand the effect of the asynchronous and synchronous methods on ROS 2 applications, we design case study 2 that contains four callback chains, each of which

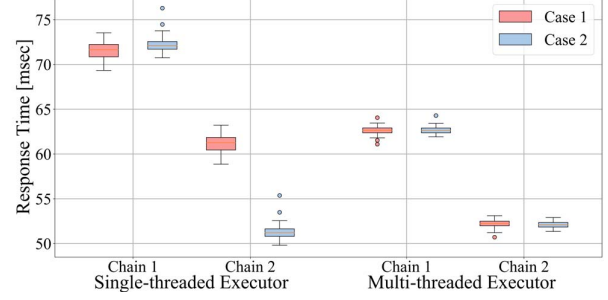


Fig. 10. Observed response time of two callback chains under different callback priority settings.

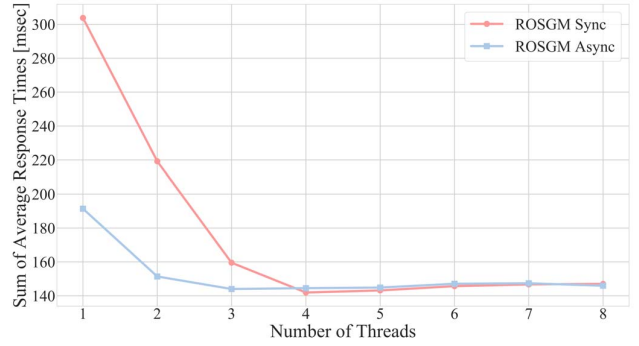


Fig. 11. Effect of the number of threads used by the executor on the response time of chains under asynchronous and synchronous methods.

has one GPU callback, as shown in Table III. We conduct experiments and calculate the sum of the average response times for the four callback chains for each experiment. In these experiments, ROSGM is configured with *Exclusive Policy*, and all GPU callbacks use the same GPU kernel, which has 224 blocks with two threads in each block.

Fig. 11 shows the observed sum of the average response times of C_1 , C_2 , C_3 , and C_4 . We use the multi-threaded executor and change the number of threads used by the executor, as shown in the x-axis of Fig. 11. Each x-axis value in Fig. 11 is the average of 1000 experiments for each method. As shown in Fig. 11, the sum of the average response times reduces as the number of threads used by the multi-threaded executor increases, and the values for the two methods tend to be equal when the number of threads reaches 4. It keeps stable with the number of threads increasing from 4 to 8. The synchronous method receives a similar or even the same performance as the asynchronous method when the number of threads used by the multi-threaded executor is equal to or greater than the number of callback chains in the application. The reason is that when the number of threads used by the multi-threaded executor is sufficient, the occupation of a CPU thread executing the GPU callback will not block other callbacks from the different callback chains. The above analysis demonstrates that ROSGM can be set with the synchronous method if the number of

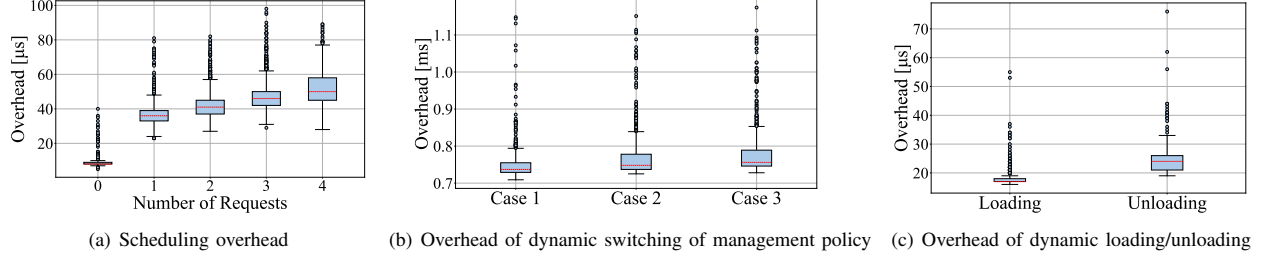


Fig. 12. The run-time overheads of ROSMG.

threads is properly configured for the multi-threaded executor.

E. Run-time Overhead

We evaluate the run-time overheads of ROSGM, including the scheduling overhead, the overhead of dynamic switching of management policy, and the overhead of dynamic task loading and unloading.

1) *Scheduling Overhead*: The scheduling overhead of ROSGM mainly comes from submitting the requests to ROSGM (i.e., the waiting queue pool) and scheduling the requests to dispatch the corresponding GPU workloads, depending on the number of requests in the waiting queues. We evaluate the scheduling overhead by recording the submission time α_1 (i.e., the time that a request is submitted to ROSGM) of a *test request* and the dispatching time α_2 (i.e., the time that the scheduler dispatches the corresponding GPU workload for execution) of it. The overhead is calculated with the result of $\alpha_2 - \alpha_1$. Fig. 12-(a) shows the scheduling overhead of ROSGM as the number of requests in front of the test request (in the same waiting queue) changes from 0 to 4. The scheduling overhead is fewer than 100 microseconds for all cases with different numbers of requests. And it increases as the number of requests in front of the test request increases.

2) *Overhead of Dynamic Switching of Management Policy*: The overhead of dynamic switching of management policy includes the overhead τ_1 for the waiting queue pool updating algorithm, the overhead τ_2 for the initialization of the newly selected policy, and the overhead τ_3 for configuring the newly selected policy in ROSGM. Among the three types of overhead, τ_1 varies with the mapping table difference between the two management policies selected before and after switching. It also varies with the number of requests that remained in the waiting queues before switching. So it is not easy to measure τ_1 by experimental evaluation. Here, we give the time complexity analysis about Algorithm 1 as the measurement of τ_1 . Suppose the number of GPU callbacks is N , and the number of the remained requests in a waiting queue is L . In Algorithm 1, there are two *for* loops and a binary search process in the second *for* loop. The loop times of the first *for* loop (line 3 to 8) is N , and the loop times of the second *for* loop (line 6 to 8) is L . The time complexity of the binary search algorithm is $\mathcal{O}(\log L)$. Hence, the time complexity of Algorithm 1 is $\mathcal{O}(NL \log L)$.

To measure the overheads τ_2 and τ_3 , we use pairwise combinations of the three management policies to form three groups, namely *Exclusive Policy* and *Sharing Policy*, *Exclusive Policy* and *Preemption Policy*, and *Sharing Policy* and *Preemption Policy*, denoted as **Case 1**, **Case 2**, and **Case 3**, respectively. We conduct 1000 experiments for each group and dynamically switch the two management policies in this group for each experiment. Assume that there is no request remaining in the waiting queues before switching. The initialization of the newly selected policy and the configuration of the newly selected policy execute in the subscription callback. We record the start time λ_1 of the callback execution and end time λ_2 of the callback execution. The overhead $\tau_2 + \tau_3$ is calculated with the result of $\lambda_2 - \lambda_1$. Fig. 12-(b) gives the resulting overheads for three groups of management policies, which are approximately between 1 ms and 2 ms.

3) *Overhead of Dynamic Task Loading and Unloading*: The overheads of dynamic task loading or unloading mainly originate from the executor's node addition or removal process. Adding or removing a node into or from the executor temporarily blocks any other scheduling behavior. That is, the executor will temporally stop scheduling the callbacks of the ROS 2 application, which leads to the overhead. Hence, the overhead is measured by recording the start time of the node addition or removal process and the end time of the node addition or removal process. We test the overhead by loading or unloading different nodes into or from a running ROS 2 application for 1000 experiments. Fig. 12-(c) shows the resulting overheads of dynamic task loading and unloading, which are both below 80 μs .

VII. RELATED WORKS

In this section, we review the literature on the real-time capability analysis and improvement of ROS 2 and then summarize related works about software-based GPU management.

A. Real-time Capability of ROS 2

The unusual scheduling strategy in the ROS 2 executor increases the unpredictability of the application. Recent works [3]–[5], [17]–[19] modeled or redesigned the executor in ROS 2 and produced the formal response time analysis of callback chains scheduled by it. For example, [5] redesigns the single-threaded executor with a priority-driven scheduler,

which allocates priorities to the callbacks according to the real-time requirements of different callback chains containing these callbacks in a ROS 2 application. In this way, they improve the predictability of an application in terms of the response time of callback chains. In [19], the authors produce a deadline-based dynamic scheduling strategy for the single-threaded executor and provide formal response time analysis for callback chains. [18] implements two new executors in ROS 2, which enables the abilities of the priorities mapping to callbacks and callback preemption for the ROS 2 application. Moreover, [17] provides an enhanced response time analysis with tighter bounds for the default ROS 2 scheduler. [20] proposes the first automatic latency management strategy for ROS 2. However, none of these works considers GPU task management for ROS 2.

To address the GPU-related issues in the first-generation ROS, [8] proposes a CPU/GPU coordination framework with an offline CPU scheduling algorithm to manage ROS nodes. It considers node-level scheduling, for which each node is assigned a priority. They apply a fixed scheduling policy for GPU management into the framework. All GPU callbacks in a node inherit the same priority as the node allocated by the CPU scheduler. At a time, only one GPU callback can launch GPU workload to GPU, which monopolizes GPU and may lead to underutilized GPU resources. The proposed policy for GPU management is fixed, which may be limited to specific applications with certain requirements.

B. Software-based GPU Management

Due to the various GPU hardware architecture, GPU management is usually complicated and hardware architecture-oriented, reducing the flexibility of the specific GPU management method. Related works focusing on GPU management can be classified into hardware-based and software-based methods. The former aims to achieve GPU management from the perspective of GPU hardware, which usually works on the GPU simulator rather than an actual GPU device [21]. The latter tries to apply the software-only mechanism for GPU management on real GPUs. Even software-based methods are usually device-dependent and application-oriented.

In the following, we summarize the existing software-based GPU management methods. In [22], [23], the authors apply the scheduling strategy using the default stream for real-time task scheduling. The existing state-of-art deep neural network frameworks, such as Caffe [24], also use only the default stream without kernel prioritization and preemption. Consequently, kernels of non-latency-sensitive tasks may delay those of latency-sensitive tasks. Some works [25]–[30] utilize multiple non-default streams to execute kernels concurrently. To minimize the application’s execution time, [31]–[33] use multiple streams to overlap kernel execution and data transfer. [34] mainly applies multiple streams and the synchronization barriers to achieve more fine-grained concurrency for DNN instances on GPU. Moreover, [22], [23], [35] provide preemption scheduling and deterministic timing guarantees by assigning different priorities to streams. In [36]–[38], SM-level scheduling strategy is used to improve quality-of-service,

system throughout, and resource utilization. [39] designs the persistent and elastic block of GPU kernels for intra-SM parallelism. More works related to block-level scheduling are presented in [40]–[42]. Besides, [43] proposes to improve system performance via a warp-level scheduling strategy.

VIII. CONCLUSION

This paper proposes a real-time GPU management framework for ROS 2 applications. In order to improve the adaptability and flexibility to meet the requirements of different execution scenarios and GPU devices, ROSGM provides a mechanism supporting the addition of any GPU management policy as a plug-in and dynamic switching among different management policies at run-time. Besides, we achieve dynamic ROS 2 task loading and unloading to the same running ROS 2 application under the management of ROSGM at run-time. By using the asynchronous GPU request submission method and proper GPU management, ROSGM can achieve considerable performance improvement for ROS 2 applications. Moreover, the flexibility of plug-in policies can better meet different requirements of different applications and even the same application under various scenarios.

ACKNOWLEDGMENT

This work is supported by the Research Grants Council of Hong Kong (GRF 11208522, 15206221) and the National Natural Science Foundation of China (NSFC 62102072, 62072085). The authors also thank the anonymous reviewers for their helpful comments.

REFERENCES

- [1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng *et al.*, “ROS: an open-source Robot Operating System,” in *ICRA workshop on open source software*, vol. 3, no. 3.2, Kobe, Japan, 2009, p. 5.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [3] D. Casini, T. Blaß, I. Lütkebohle, and B. Brandenburg, “Response-time analysis of ROS 2 processing chains under reservation-based scheduling,” in *31st Euromicro Conference on Real-Time Systems*. Schloss Dagstuhl, 2019, pp. 1–23.
- [4] Y. Tang, Z. Feng, N. Guan, X. Jiang, M. Lv, Q. Deng, and W. Yi, “Response time analysis and priority assignment of processing chains on ROS2 executors,” in *2020 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2020, pp. 231–243.
- [5] H. Choi, Y. Xiang, and H. Kim, “PiCAS: New design of priority-driven chain-aware scheduling for ROS2,” in *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2021, pp. 251–263.
- [6] S. Kato, E. Takeuchi, Y. Ishiguro, Y. Ninomiya, K. Takeda, and T. Hamada, “An open approach to autonomous vehicles,” *IEEE Micro*, vol. 35, no. 6, pp. 60–68, 2015.
- [7] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kit-sukawa, A. Monroy, T. Ando, Y. Fujii, and T. Azumi, “Autoware on board: Enabling autonomous vehicles with embedded systems,” in *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPs)*. IEEE, 2018, pp. 287–296.
- [8] Y. Suzuki, T. Azumi, S. Kato, and N. Nishio, “Real-Time ROS extension on transparent CPU/GPU coordination mechanism,” in *2018 IEEE 21st International Symposium on Real-Time Distributed Computing (ISORC)*. IEEE, 2018, pp. 184–192.
- [9] V. Mayoral-Vilches, S. M. Neuman, B. Plancher, and V. J. Reddi, “RobotCore: An Open Architecture for Hardware Acceleration in ROS 2,” *arXiv preprint arXiv:2205.03929*, 2022.

- [10] C. Yu, Y. Bai, H. Yang, K. Cheng, Y. Gu, Z. Luan, and D. Qian, "Smguard: A flexible and fine-grained resource management framework for GPUs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 12, pp. 2849–2862, 2018.
- [11] Q. Sun, Y. Liu, H. Yang, Z. Luan, and D. Qian, "Smqos: Improving utilization and energy efficiency with qos awareness on GPUs," in *2019 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2019, pp. 1–5.
- [12] ROS 2. (2020) Single-threaded executor. [Online]. Available: https://github.com/ros2/rclcpp/blob/foxy/rclcpp/src/rclcpp/executors/single_threaded_executor.cpp
- [13] —. (2020) Multi-threaded executor. [Online]. Available: https://github.com/ros2/rclcpp/blob/foxy/rclcpp/src/rclcpp/executors/multi_threaded_executor.cpp
- [14] NVIDIA. (2019) CUDA stream. [Online]. Available: https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__STREAM.html
- [15] T. Amert, N. Otterness, M. Yang, J. H. Anderson, and F. D. Smith, "GPU scheduling on the NVIDIA TX2: Hidden details revealed," in *2017 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2017, pp. 104–115.
- [16] C. Bédard, I. Lütkebohle, and M. Dagenais, "ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6511–6518, 2022.
- [17] T. Blaß, D. Casini, S. Bozhko, and B. B. Brandenburg, "A ROS 2 Response-Time Analysis Exploiting Starvation Freedom and Execution-Time Variance," in *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2021, pp. 41–53.
- [18] C. Randolph, "Improving the Predictability of Event Chains in ROS 2," Master's thesis, Delft University of Technology, 2021.
- [19] A. A. Abdullah, V. Sudharsan, M. W. Kurt, S. Jinghao, and G. Zhishan, "Response Time Analysis for Dynamic Priority Scheduling in ROS2," in *In Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC '22)*. IEEE, 2022.
- [20] T. Blass, A. Hamann, R. Lange, D. Ziegenbein, and B. B. Brandenburg, "Automatic Latency Management for ROS 2: Benefits, Challenges, and Open Problems," in *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2021, pp. 264–277.
- [21] C. Zhao, W. Gao, F. Nie, and H. Zhou, "A Survey of GPU Multitasking Methods Supported by Hardware Architecture," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 6, pp. 1451–1463, 2021.
- [22] H.-E. Zahaf and G. Lipari, "Design and analysis of programming platform for accelerated GPU-like architectures," in *Proceedings of the 28th International Conference on Real-Time Networks and Systems*, 2020, pp. 1–10.
- [23] N. Feddal, H.-E. Zahaf, and G. Lipari, "Toward Precise Real-Time Scheduling on NVidia GPUs," in *15th Junior Researcher Workshop on Real-Time Computing 2022*, 2022, p. 20.
- [24] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, "Caffe: Convolutional architecture for fast feature embedding," in *Proceedings of the 22nd ACM international conference on Multimedia*, 2014, pp. 675–678.
- [25] Y. Wang and H. Kim, "Work-in-progress: Understanding the effect of kernel scheduling on GPU energy consumption," in *2019 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2019, pp. 584–587.
- [26] S.-K. Shekofteh, H. Noori, M. Naghibzadeh, H. Fröning, and H. S. Yazdi, "ccuda: Effective co-scheduling of concurrent kernels on GPUs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 4, pp. 766–778, 2019.
- [27] W. Kwon, G.-I. Yu, E. Jeong, and B.-G. Chun, "Nimble: Lightweight and parallel GPU task scheduling for deep learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 8343–8354, 2020.
- [28] D.-L. Lin and T.-W. Huang, "Efficient GPU computation using task graph parallelism," in *European conference on parallel processing*. Springer, 2021, pp. 435–450.
- [29] G. Gilman, S. S. Ogden, T. Guo, and R. J. Walls, "Demystifying the placement policies of the NVIDIA GPU thread block scheduler for concurrent kernels," *ACM SIGMETRICS Performance Evaluation Review*, vol. 48, no. 3, pp. 81–88, 2021.
- [30] M. Han, H. Zhang, R. Chen, and H. Chen, "Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 539–558.
- [31] K. Raju and N. N. Chiplunkar, "Performance enhancement of CUDA applications by overlapping data transfer and Kernel execution," *Applied Computer Science*, vol. 17, no. 3, 2021.
- [32] A.-J. Lázaro-Muñoz, B. López-Albelda, J. M. González-Linares, and N. Guil, "A Scheduling Theory Framework for GPU Tasks Efficient Execution," in *International Conference on Vector and Parallel Processing*. Springer, 2018, pp. 146–159.
- [33] A. Lázaro-Muñoz, J. M. González-Linares, J. Gómez-Luna, and N. Guil, "A tasks reordering model to reduce transfers overhead on GPUs," *Journal of Parallel and Distributed Computing*, vol. 109, pp. 258–271, 2017.
- [34] F. Yu, S. Bray, D. Wang, L. Shangguan, X. Tang, C. Liu, and X. Chen, "Automated Runtime-Aware Scheduling for Multi-Tenant DNN Inference on GPU," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.
- [35] Y. Xiang and H. Kim, "Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference," in *2019 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2019, pp. 392–405.
- [36] S. K. Saha, Y. Xiang, and H. Kim, "STGM: Spatio-temporal GPU management for real-time tasks," in *2019 IEEE 25th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2019, pp. 1–6.
- [37] S. Jain, I. Baek, S. Wang, and R. Rajkumar, "Fractional GPUs: Software-based compute and memory bandwidth reservation for GPUs," in *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2019, pp. 29–41.
- [38] A. Zou, J. Li, C. D. Gill, and X. Zhang, "RTGPU: Real-Time GPU Scheduling of Hard Deadline Parallel Tasks with Fine-Grain Utilization," *arXiv preprint arXiv:2101.10463*, 2021.
- [39] H. Zhao, W. Cui, Q. Chen, J. Zhao, J. Leng, and M. Guo, "Exploiting Intra-SM Parallelism in GPUs via Persistent and Elastic Blocks," in *2021 IEEE 39th International Conference on Computer Design (ICCD)*. IEEE, 2021, pp. 290–298.
- [40] Q. Chen, H. Chung, Y. Son, Y. Kim, and H. Y. Yeom, "Smcompactor: a workload-aware fine-grained resource management framework for GPGPUs," in *Proceedings of the 36th Annual ACM Symposium on Applied Computing*, 2021, pp. 1147–1155.
- [41] Z. Ji and C.-L. Wang, "Collaborative GPU Preemption via Spatial Multitasking for Efficient GPU Sharing," in *European Conference on Parallel Processing*. Springer, 2021, pp. 89–104.
- [42] S. Kim and Y. Kim, "K-Scheduler: dynamic intra-SM multitasking management with execution profiles on GPUs," *Cluster Computing*, vol. 25, no. 1, pp. 597–617, 2022.
- [43] T. T. Yeh, A. Sabne, P. Sakdhnagool, R. Eigenmann, and T. G. Rogers, "Pagoda: A GPU runtime system for narrow tasks," *ACM Transactions on Parallel Computing (TOPC)*, vol. 6, no. 4, pp. 1–23, 2019.